UNIT 5 · TOPIC 5.5

Secure Web Application Design

Assignment

Instructions: 15 practice questions + 3 coding assignment(s), 30 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

5.5.a · Input validation and sanitization

1. Which validation strategy defines exactly what valid input looks like and rejects everything else, rather than trying to enumerate every bad pattern? (1 pt)

- ○ A. Denylist (blocklist)
- ○ B. Allowlist (whitelist)
- ○ C. Output encoding
- ○ D. Canonicalization

2. Select ALL statements that correctly describe why client-side (JavaScript) validation is not a security control. (3 pts)

- □ A. A browser is entirely under the attacker's control, so JavaScript checks can be disabled or bypassed
- □ B. An attacker can send a crafted request directly to the server with a tool like curl, skipping the browser entirely
- □ C. Client-side validation must still be re-enforced on the server for any security-relevant check
- □ D. Client-side validation is cryptographically enforced and cannot be bypassed

(Select ALL that apply.)

3. An application validates a raw, un-decoded path string before checking it for '../' sequences. Why is this a mistake? (1 pt)

- ○ A. Checking the raw string is fine, as long as the check is done on the server
- ○ B. The check should be done twice on the raw string to be safe
- ○ C. An attacker can URL-encode the traversal sequence (e.g. ..%2F) so it slips past the raw-string check, then gets decoded later, this is a canonicalization bypass
- ○ D. Raw strings are always rejected by web servers before reaching the app

4. Name the security concept that transforms data at the point it is inserted into a new context (HTML, SQL, shell) so special characters are treated as literal data, not code. (1 pt)

5. SQL injection, cross-site scripting (XSS), and command injection share what root cause? (1 pt)

- ○ A. They are all caused by weak passwords on the server

○ B. The application trusted that input was 'just data' when a downstream interpreter treated part of it as code or control syntax

○ C. They all exploit unencrypted connections between client and server

○ D. They all happen because the server runs an outdated operating system

5.5.b · Secure coding practices

6. Which practice eliminates SQL injection risk regardless of what input validation is or isn't done upstream? (1 pt)

○ A. Parameterized queries

○ B. Escaping single quotes in user input

○ C. Using a web application firewall

○ D. Validating input against an allowlist

7. Select ALL reasons secrets (API keys, passwords) should NOT be hardcoded in source code. (3 pts)

□ A. They remain visible in version control history forever, even if later removed

□ B. Rotating a compromised secret requires a code change and redeployment

□ C. They are automatically encrypted by the version control system

□ D. Anyone with repository read access can see them

(Select ALL that apply.)

8. An authentication check throws an unexpected exception. Secure coding says the application should: (1 pt)

○ A. Retry the check until it succeeds

○ B. Allow access but log a warning for review

○ C. Fall back to a simpler check, such as username only

○ D. Deny access by default (fail securely/fail closed)

9. Name the principle that says application processes and database connections should run with only the minimum access they need to function. (1 pt)

10. Why is pinning and patching third-party dependency versions considered a secure-coding practice, not just a maintenance task? (1 pt)

○ A. A vulnerability in any dependency becomes a vulnerability in the whole application, so unpatched dependencies are unpatched attack surface

○ B. Pinning versions makes the app download faster

○ C. Unpinned dependencies are blocked by most firewalls

○ D. Dependencies run in a sandbox, so only their bugs that crash the app matter

5.5.c · Application security testing (SAST/DAST)

11. Which testing approach reads an application's source code without executing it? (1 pt)

○ A. DAST

○ B. Fuzzing only

○ C. Penetration testing

○ D. SAST

12. Select ALL statements that correctly describe the tradeoffs between SAST and DAST. (3 pts)

□ A. SAST can run before the code is even merged, giving very early feedback

□ B. DAST requires a running, reachable instance of the application

□ C. SAST tends to produce more false positives because it cannot always confirm attacker-controlled input reaches a flagged line

□ D. DAST can see every code path in the application, including rarely-used admin features

(Select ALL that apply.)

13. A dynamic scanner sends a crafted request to a live login form and observes that it can bypass authentication. What kind of finding is this? (1 pt)

○ A. SAST-only, since it's about code logic

○ B. DAST, since it required interacting with the running application

○ C. Neither, since automated tools cannot find this

○ D. A compiler warning

14. Name the kind of security testing that is performed by a skilled human who chains findings together and reasons about business logic flaws automated SAST/DAST tools cannot recognize. (1 pt)

15. Why do mature CI/CD pipelines typically run BOTH SAST and DAST rather than picking just one? (1 pt)

○ A. SAST is required for compliance, while DAST is optional

○ B. DAST finds everything SAST finds, so SAST is only for speed

○ C. They find largely non-overlapping vulnerability classes, so together they give broader coverage

○ D. Running both lets each tool double-check the other's false positives

Coding Assignments (Python 3)

5.5.a · Input validation and sanitization

3 pts · auto-graded on the site

Coding problem 5.5.a: Field validator (allowlist + injection detector)

Skill: Mitigate Risk · **Maps to:** 5.5.A · **Language:** Python 3 (pytest)

Problem

Build the server-side validation gate described in the lesson: given a batch of submitted form field values, classify each as SAFE or BLOCKED using **allowlist rules per field type** plus detection of known **injection meta-characters/keywords**, the same two checks a real input-validation layer runs before any value reaches a database query, HTML page, or shell.

Function contract

Implement `solve(submissions: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`submissions`: `list[dict]`, each `{"field": str, "type": str, "value": str}`. `type` is one of `"zip"`, `"email"`, `"quantity"`, `"username"`.

Output

`list[str]` of the `value`s that are BLOCKED, in the same order they appeared in the input.

Allowlist rules (per `type`)

- `zip`: exactly 5 digits.
- `email`: `local@domain.tld` shape.
- `quantity`: integer string 1-99.
- `username`: 3-20 letters/digits/underscore.

A value is also BLOCKED (even if it matches the allowlist shape) if it contains a known injection signature: `--`, `;`, `<script`, `UNION SELECT`, `OR 1=1`, `' OR '`, `DROP TABLE`, `../`, `%3B`, `%2F` (case-insensitive).

Examples

```
[{"field":"zip","type":"zip","value":"94107"}]                    ->  []
[{"field":"zip","type":"zip","value":"9410"}]                     ->  ["9410"]
[{"field":"u","type":"username","value":"admin'--"}]               ->  ["admin'--"]
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 5.5.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Dict, List


def solve(submissions: List[Dict[str, str]]) -> List[str]:
    """Classify each submission against per-field allowlist rules and known injection
    meta-character signatures; return the `value`s that are BLOCKED, in input order.

    Args:
        submissions: list of {"field": str, "type": str, "value": str}. `type` is one of
            "zip", "email", "quantity", "username".
    Returns:
        list[str] of the `value`s that fail validation (blocked), preserving input order.
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

5.5.b · Secure coding practices

3 pts · auto-graded on the site

Coding problem 5.5.b: Secure-coding configuration auditor

Skill: Mitigate Risk · **Maps to:** 5.5.A, 5.5.B · **Language:** Python 3 (pytest)

Problem

Build a lightweight policy-as-code auditor: given an application's configuration, flag every secure-coding practice it violates, exactly the checks a code reviewer or CI gate would run.

Function contract

Implement `solve(config: dict) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`config`: `dict` of settings. Any key may be missing.

Output

Sorted `list[str]` of violated rule names.

Rule set

| condition | violation | |---|---| | `debug_mode is True` | `"debug-mode-enabled"` | | `db_query_style == "concatenated"` | `"non-parameterized-query"` | | `secrets_in_code is True` | `"hardcoded-secret"` | | `error_verbosity == "verbose"` | `"verbose-error-messages"` | | `dependencies_pinned is False` | `"unpinned-dependencies"` | | `runs_as == "root"` | `"excessive-privilege"` | | `code_review_required is False` | `"no-code-review"` |

Examples

```
{"debug_mode": True}                          ->  ["debug-mode-enabled"]
{"db_query_style": "parameterized"}           ->  []
{"runs_as": "root", "secrets_in_code": True} ->  ["excessive-privilege", "hardcoded-secret"]
```

Constraints

- Deterministic, offline, standard library only.
- A missing key never triggers a violation.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 5.5.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Dict, List


def solve(config: Dict[str, object]) -> List[str]:
    """Audit an application configuration against the secure-coding rule set.

    Args:
        config: dict of settings, e.g. {"debug_mode": bool, "db_query_style": str,
            "secrets_in_code": bool, "error_verbosity": str, "dependencies_pinned": bool,
            "runs_as": str, "code_review_required": bool}. Any key may be missing.
    Returns:
        sorted list[str] of violated rule names. See interface.md for the exact rule set.
    """
```

```
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

5.5.c · Application security testing (SAST/DAST)

3 pts · auto-graded on the site

Coding problem 5.5.c: Static-analysis scanner (SAST)

Skill: Detect Attacks, Mitigate Risk · **Maps to:** 5.5.B, 5.5.C · **Language:** Python 3 (pytest)

Problem

Build a miniature SAST rule engine. Given the source lines of a file (as a list of strings, one per line, 1-indexed), scan each line for known dangerous patterns and return a list of findings, exactly what a real static-analysis tool's rule engine does before a human triages the results.

Detect these categories:

- `"sql-injection-risk"`: a SQL keyword (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) combined with string concatenation (`+`) or an f-string/`.format(`/`%` on the same line (i.e., the query is being built dynamically instead of parameterized).
- `"eval-use"`: the line calls `eval(` or `exec(`.
- `"hardcoded-secret"`: the line assigns a literal string to a variable whose name contains `password`, `secret`, `api_key`, or `token` (case-insensitive), e.g. `password = "hunter2"`. An assignment that reads from an environment variable or config object (no literal string on the right-hand side) is NOT a finding.

Function contract

Implement `solve(lines: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `lines`: `list[str]`, source code lines, in file order. Line numbers are 1-indexed (the first element of the list is line 1).

Output

A `list[dict]`, each `{"line": int, "category": str}`, **sorted by line number ascending**, then by category name ascending for findings on the same line. If a line matches more than one category, emit one finding per matching category. If no lines match, return `[]`.

Examples

```
lines = [
  'query = "SELECT * FROM users WHERE name = \'" + user_input + "\'"',
  'password = "hunter2"',
  'eval(user_expr)',
]
solve(lines) -> [
  {"line": 1, "category": "sql-injection-risk"},
  {"line": 2, "category": "hardcoded-secret"},
  {"line": 3, "category": "eval-use"},
]
```

```
lines = ['password = os.environ["DB_PASSWORD"]', 'db.query(stmt, params)']
solve(lines) -> []
```

Constraints

- Deterministic, offline, standard library only.

- Findings are sorted by `line`, then `category` (both ascending).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 5.5.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Dict, List


def solve(lines: List[str]) -> List[Dict[str, object]]:
    """Scan source lines for known dangerous patterns (miniature SAST rule engine).

    Args:
        lines: list of source-code line strings, 1-indexed by position.
    Returns:
        list of {"line": int, "category": str} findings, sorted by line then category
        (both ascending); a line may contribute more than one finding. See prompt.md for
        the exact category rules.
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```